

Next: 以直接担保实现连续 UTXO 支付

补充材料（中文阅读版）

Lu Hengrun

S1. 原始执行关联的细节

最早的冲突 polka。固定一个高度，以及第 r 轮中针对非空值 v 的一份提交证书，并从该证书中选出两名诚实签署者 K 。每名签署者在发出其 `precommit` 之前先锁定 v 。证书可以在这些签署者已经推进之后才汇集，所以它被观察到的时间无关紧要。

假设存在针对不同值的更高轮 `polka`（若 `nil polka` 允许解锁，则也计入）。取存在这种 `polka` 的最小轮次 $s > r$ ，并在 s 内取其不同的 `prevote` 全部存在的最早事件 τ 。其三名投票者中包含某个 $m \in K$ 。诚实节点的轮次递增，所以 m 在第 s 轮的 `prevote` 晚于它在第 r 轮的 `precommit`，而没有解锁理由时， m 必须对其已锁定的值做 `prevote`。在中间轮次中重新锁定 v ，并不改变这一约束。来自 s 以下某一轮的冲突理由，与 s 的选取相矛盾；来自第 s 轮的冲突理由，必然在 m 投出 τ 时被计入的那一票之前就已存在，这与 τ 的选取相矛盾。来自 s 之上的证据，不可能先于第 s 轮中较晚的投票。因此这样的 `polka` 不存在，而更高轮的冲突提交必然需要它。同一轮中冲突的提交，则需要某个诚实节点做出两次 `precommit`。这一论证依赖于所述的锁定转移规则和轮次单调性规则，并不要求及时收到提交证据。

配对、验证与执行。对候选更新做归纳。本地构造把区块体与其原始分片配对，远程补全则由经检查的分片重建区块体。在 `Locked` 或 `Valid` 值移动时，二者一并转移；切换目标则清除旧区块体。历史安装不触及这些活跃候选。非空的 `precommit` 紧随匹配的 `polka`。诚实的 `prevote` 在实际的提案验证或更早的锁定中有其理由，追溯后者时锁定轮次逐步降低，直到抵达验证。最后的完整身份守卫在 `Finalize` 之前固定原始交易、高度、时间和其他应用上下文。同步与重放提供相同的原始材料，所以在这一调用边界之内，同一高度的重试不会通过稳定哈希缓存引入第二个原始区块体。

写集。正文中的证明依赖于下列写集。赔付决定写入：签发组织备付金账户、义务状态、`Promise` 与覆盖记录（`Paid` 和 `Remaining`）、授权额度用量（`Spent`）、公共缺口、消费方的支付记录（`Pending` 和费用阶段）、奖励与退款余额及费用输出、到期索引条目的移除，以及决定记录及其决定索引条目和待历史表示条目。赔付之后源交易的执行写入偿还：签发组织备付金账户、义务状态（`Recovered`）、覆盖记录（`Recovered`）和授权额度用量（`Spent`）。它不改变决定记录。补交任务只写入成员本地的 `outbox`。历史表示命令或批处理写入逻辑修订、头命令、任务或批处理任务及其逐输出引用、队列条目，以及待历史表示条目的移除。安装只写入本地区块存储的版本及其自身的进度游标。历史表示和安装的写入都不属于经济投影。

固定目标集合的收敛性。固定一个原始区块体和一组已决定槽位。每个槽位的替换消息是网络和输出的函数，因此无论如何打包，都以相同的消息替换相同的槽位。若变色龙关系对每个固定的消息与承诺都有唯一的有效 `opening`（参数正确生成且公开指数与群的阶互素时即成立），则每个槽位 `opening` 和每个分片 `opening` 都是其关系的唯一解。分片上下文取决于高度和分片索引，与修订计数器无关。因此，只要覆盖该集合，历史表示命令的任何分组或顺序都得到相同的区块体和分片 `opening`。单条目命令、单个批处理以及中间规模的批处理，在命令标识、修订号和应用哈希上各不相同。该陈述针对的是最终字节，并如上所述依赖于参数生成和编码。

跟随者更新。对于从相同的本地状态、配置和游标出发的跟随者，只要两种表示都能解码，相同的固定支付字段与经认证的结果就诱导出相同的经济更新。可变的资金引用与 `opening` 不选择任何 `Paid`、`Pending` 或费用效果。决定被完整认证，执行结果承诺则认证其效果和费用输出。因此，经济观察与对最新可变历史区块体的独立验证保持分离。

分析中使用的关系。三种关系比较的是不同的对象。历史表示的经济惰性，比较一次历史表示或安装步骤前后的经济投影，并投影掉命令标识、修订计数器和物化任务。固定目标收敛性，比较不同的历史表示命令分组与顺序为同一原始区块体和同一已决定集合所产生的最终区块体和分片 `opening`。重放相等性固定整个原始命令序列及其区块上下文，确定性执行随后重现每个应用哈希。跨历史比较固定经济命令、公共时间和历史目标坐标，因此投影相等并不能确定整个应用状态相同，也不允许删除区块或给区块重新编号。这三种关系都不比较以不同顺序安排父交易到达与赔付的历史；正文的延迟中性定理在其自身的假设下做这一比较，它等同的是最终的 `CAL` 持有者和备付金余额，而不是应用哈希或修订元数据。

S2. 最终版本逐轮证据

本节记录正文使用的生产版本 721800c。连续支付和混合负载使用相同节点二进制及 `Comet` 源码。以下历史方法和旧版本表格保留各自构建身份，不为此版本的吞吐测量。

表 S1
最终版本连续支付；各时间自首笔实际发送起计，单位秒。

运行	未跳收款	全链公共观察	成员关闭	尝试	缺口
r1-fast	4.497998	5.228984	5.293857	100	3
r1-wait	64.626603	65.232083	65.284395	149	0
r2-fast	4.739972	5.253720	5.295722	100	2
r2-wait	64.475560	65.105699	65.154442	116	0
r3-fast	4.556805	5.114055	5.129723	100	4
r3-wait	64.460699	65.075768	65.124437	142	0

这些成本来自离线成功执行夹具。表示维护包含顺序适配、组装、`Finalize`、同步应用 `Commit` 与物化；共同赔付决定单列。网络、共识等待、公共块保存和提交签名构造不在此计时内。`KV` 是真实编码记录的逻辑增量，不是两种独立部署的物理磁盘空间差。

S3. 归档构建的实验方法

版本谱系与部署。结果来自三类版本（表 S4）。续花与记账基线以提交 `d99ec92` 为准，实现分层续花和记账转换；其赔付与历史表示一并提交。制品记录了每个二进制文件的 `SHA-256` 指纹和每次运行的配置。九服务的续花、赔付与偿还、存储和短时吞吐各系列使用该版本。最初的十四服务配对运行使用相同的单义务规则，其时批处理结果标签尚未与直接结果标签分开，跨类型编解码回归测试覆盖了这一分离。基线的存储、内存续花和短时吞吐对照中的成员以 `GOGC=200` 和 `GOMAXPROCS=16` 运行；该覆盖设置不用于委员会、网关或钱包进程。

decision-first 版本（记账域 `DIRECT_ACCOUNTING_V5_DECISION_FIRST`）实现基于公共证书的补交、赔付决定，以及独立的历史表示命令。它源自分支基点 `0e7bae5`，后者是 `d99ec92` 的仅含文档变更的后代。九次功能运行使用初始实现快照，其源码包 `SHA-256` 以 `2c91b0251024322c` 开头。该系列后期快照的包 `SHA-256`

表 S2

混合负载逐轮普通付款；延迟单位 ms，跨度和排空单位 s。后两列为总未完成上限和快速并发上限命中计数。

运行	收款 P50	收款 P95	派发 P95	发送跨度	排空	总限	快限
r1-C	53.435	136.221	201.339	70.191	1.029	1	0
r1-N	55.259	152.766	255.396	70.245	1.015	16	0
r1-P	55.102	154.780	895.879	70.886	1.356	63	935
r1-R	53.034	139.894	0.239	69.990	0.815	0	0
r2-C	54.011	136.490	700.329	70.690	0.906	68	760
r2-N	53.653	143.869	110.389	70.416	0.820	18	0
r2-P	52.892	134.963	0.242	69.990	1.066	0	0
r2-R	53.832	144.903	0.218	69.990	1.125	0	0
r3-C	55.682	159.437	405.296	70.395	1.046	65	2
r3-N	55.132	155.992	0.242	69.977	1.200	5	0
r3-P	54.996	144.206	586.541	70.576	1.155	81	549
r3-R	52.706	126.355	0.186	69.984	0.890	0	0

表 S3

历史表示逐进程成本；维护及共同决定执行/提交单位 ms，净新增 KV 单位字节。

进程	付款	赔付	表示维护	净新增 KV	共同决定
1	32	1	43.322	292136	8.926
2	32	1	43.664	292200	8.917
3	32	1	42.035	292162	9.013
1	32	8	97.183	329118	11.919
2	32	8	94.461	329126	11.984
3	32	8	93.591	329059	12.029
1	256	1	64.826	2258078	9.052
2	256	1	66.931	2258044	9.989
3	256	1	65.323	2258085	8.993
1	256	8	214.398	2294947	20.119
2	256	8	213.902	2294910	21.027
3	256	8	214.781	2294980	20.967

以 449b4d4072c99588 开头，它在读取任何历史区块之前就拒绝位置与已提交状态不匹配的决定，并把每轮到期决定的有界扫描从 8 项提高到 32 项；普通支付路径不变。六次普通路径运行、完整测试套件和竞态测试使用该系列后期快照。经审计的提交 f185f73 另外要求附在支付上的证书与其证书输入一一对应，它不改变所测负载中被接受的支付。基线版本与 decision-first 版本的数值分开报告。

归档的运行范围运行记录基点 e3870e1 与实现 a083ff6。原始续花记录 f0c0a274，资本记录基点 8e1fb40 与实现 5807546，组织路径记录基点 bb65af69 与已验证源码 13be856。归档的批处理集成使用 f82f8cd，结构性历史表示计数则在基线代码上重复测得。源交易失败检查点 c1ebf2d 使用已被取代的迟到源交易规则，不支持当前的论断。

表 S4
各版本及其所支持的系列。

版本	标识	系列
续花与记账基线	d99ec92	100 跳链、同步成员与存储对照、赔付与偿还矩阵、配对干扰运行、短时吞吐。
Decision-first	基点 0e7bae5; 功能运行包 2c91b0251024322c; 该系列后期快照 449b4d4072c99588; 经审计提交 f185f73	九次功能运行、六次普通路径运行、回归测试与竞态测试。
归档	e3870e1/a083ff6、f0c0a274、8e1fb40/5807546、bb65af69/13be856、f82f8cd、c1ebf2d	运行范围、原始续花、资本、组织路径、批处理集成、源交易失败检查点。

Mac 试验共用一台 M4 Max 主机，16 个 CPU 核心、64 GiB 内存，结构性微基准测试运行于 Windows amd64。基线版本的内存链和短时吞吐试验使用内存中的服务状态和 NoSync 钱包。赔付与偿还试验使用落盘的委员会应用状态和 BlockStore，成员和网关在内存中。存储对照只改变成员提交，在 bbolt NoSync 与同步事务之间切换。每次运行都从全新的创世状态开始。另有六次十四服务的落盘链运行，使用基线记账规则和用户自付 FUEL。其中三次快速 100 跳运行耗时 668.960–688.080 ms，跳间确认运行耗时 114.496–118.742 s。它们属于与正文九服务结果不同的存储与部署配置。

Decision-first 功能运行。九次运行使用十四服务配置，450 个初始输出，落盘的委员会，成员为默认的 NoSync 模式，flush 与 gossip 设置为 10 ms，显式授权的组织代付 FUEL；签名及所有金额与消费检查保持开启。普通支付压力钱包使用 NoSync 钱包选项，两跳故障演示使用默认钱包模式。每次运行中，一个注入的故障使原发布者在完整 QC 形成之后无法提交或 INSTALL 父支付，所以原签署者持有其批准而不持有 QC。子支付花费该父支付的 TXCer。在补交处理中，驱动程序从不提交原支付。对照处理通过实验开关关闭自动补交步骤。持有经 INSTALL 存入的完整材料的成员所做的普通备用提交仍保持启用；INSTALL 本身负责在组织内复制该材料。暂停处理同样关闭补交，并且在标记文件存在期间，使全部四名委员会成员的输入适配与分片适配 HTTP 端点返回暂停状态；共识、决定和支付处理并未暂停。

计时从子支付第一次 HTTP 发送开始。快速接收在钱包验证证书并记录输出时结束。公共事件通过轮询跟随者的状态端点来观察，所以包含轮询延迟，两次公共观察之差不是从确切提交时刻起算的协议延迟。赔付时间包含配置的 30-s 义务期、委员会 tick 推进的公共时间，以及后台调度。在暂停处理中，脚本在赔付之后提交迟到的源交易，并等待全部四名委员会成员的状态端点都显示义务已偿还、且历史表示既未提交也未物化。它保存这四项观察，移除标记，然后等待历史表示在全部四名成员处提交并物化。状态端点的稳定性标志比较头部哈希、数据哈希和交易身份。完整区块身份与分片集合头由物化步骤和真实共识的集成测试检查，所以不把该标志单独当作完整身份的证明。偿还以及适配的恢复没有各自的时间戳。这些测量不包含与周期性扫描、投票收集，或暂停适配下的早期耦合版本的比较。

每种处理有三轮，每次功能运行都通过四项检查：四个委员会成员的状态哈希相同、缺口为零、支付已关闭、outbox 已排空。补交与对照两种处理还各自带有三次各 300 笔普通支付的运行。逐次运行的数值见表 S5。

Decision-first 普通路径运行。六次运行比较位于分支基点的版本与最终的 decision-first 快照，顺序为 AB、

表 S5

九次 DECISION-FIRST 功能运行的逐次记录。时间是从子支付的钱包发送起算的客户端观察；“父支付快速接收”是原支付自身的接收。公共执行各列包含轮询。破折号表示该列不适用或该时间未被记录。在对照与暂停各行中，“自动补交源交易”一列不适用，因为那里关闭了自动补交；驱动程序随后提交该源交易，它得以执行并偿还签发组织（由偿还审计验证），但该时刻没有加时间戳。在补交各行中，没有发生赔付或物化。

处理	运行	子支付快速接收 (ms)	父支付快速接收 (ms)	子支付公共执行 (s)	自动补交源交易 (s)	赔付 (s)	物化 (s)
补交	1	13.94	9.02	1.414	2.014	—	—
	2	13.94	8.94	1.990	2.565	—	—
	3	13.91	9.02	1.440	2.039	—	—
对照	1	11.96	8.97	1.364	—	35.839	36.994
	2	12.94	8.97	1.413	—	35.888	37.021
	3	12.01	9.00	1.388	—	35.864	37.019
暂停	1	12.00	7.98	1.362	—	35.837	42.748
	2	13.92	9.04	1.341	—	35.814	42.736
	3	13.92	9.03	1.841	—	35.891	42.798

表 S6

六次普通路径运行的逐次记录。运行顺序为 AB、BA、AB（A = 分支基点，B = DECISION-FIRST）。分位数针对单次运行的 300 笔支付。

版本	运行	总耗时 (s)	快速 P50 (ms)	快速 P95 (ms)	区块 P50 (ms)	派发 P95 (ms)	发送时段 (s)
分支基点	1	3.871	1.112	27.441	1151.0	0.338	2.990
	2	3.815	1.096	17.951	1101.1	0.295	2.990
	3	3.805	1.052	39.945	1076.0	0.437	2.990
Decision-first	1	4.000	1.071	30.530	1127.9	0.430	2.990
	2	3.845	1.073	22.084	1103.9	0.355	2.990
	3	3.791	1.045	33.744	1052.3	0.383	2.990

BA、AB。每次以 100 笔/秒发送 300 笔支付，并发快速请求 64 个，未完成请求至多 256 个，使用 NoSync 压力钱包。总耗时包含全部 300 笔支付的公共确认和成员跟随；它既不是单笔支付的时间，也不是最大吞吐。每次运行的发送时段约为 2.990s。正文中的汇总数值是各次运行统计量的中位数，而不是对全部支付合并计算的分位数。普通的最终输出不携带输入证书，所以这些运行没有测量输入尚未成为最终输入 of 支付所保留的存储。逐次运行的数值见表 S6。

回归测试覆盖。赔付决定在尚未到期、网络/槽位/金额不匹配、备付金不足，或与已记录的决定冲突时被拒绝。重复的决定没有经济效果，包括偿还之后，此时它不返回任何变更、效果或费用输出；源交易先到与决定先到两种顺序都已覆盖。补交针对跨组织和多层源交易、重复暴露，以及已被观察到或已排队的支付（这些支付不会被重建）做了测试。普通验证拒绝不匹配的材料，补交前后原始批准、支付字节和 CAL 额度保持不变。历史表示要求决定已成功，在源交易偿还之后仍然可以进行，并且不会第二次恢复成员进度或 Worker 切片。在身份与重放方面，真实的门限适配保持完整共识身份、后继交易的区块体和预先签名的后继交易引用。对原始历史的重放在每个高度重现应用状态，对同一已决定集合逐项和不同分组的历史表示得到相同的

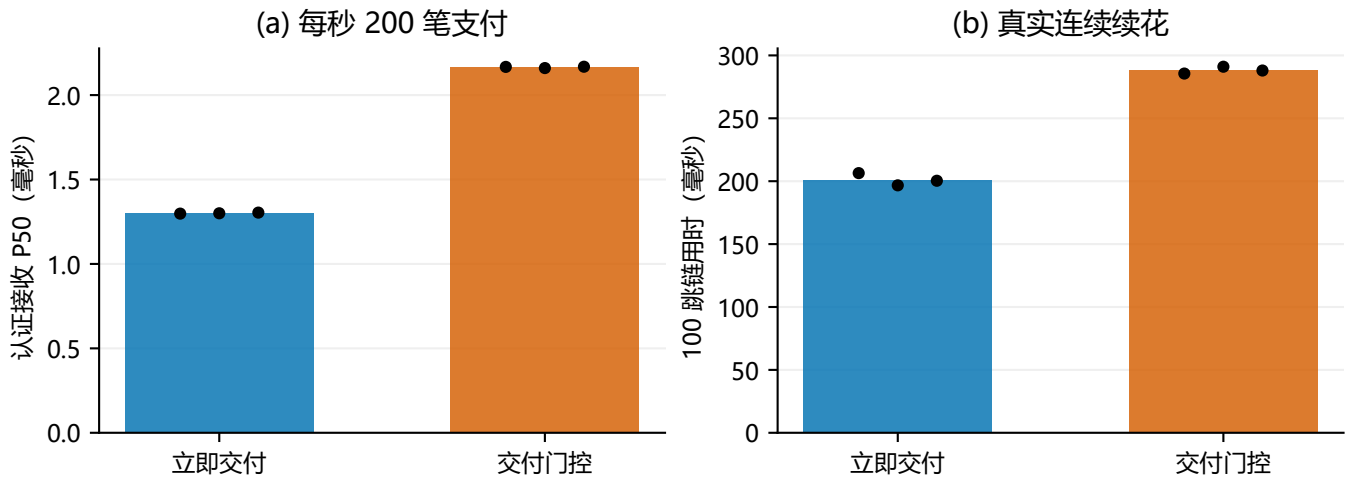


图 S1. 交付门控的效果 (ms)。左：每秒 200 笔支付下的运行级接收 P50，每次运行 6,000 笔支付，三次配对重复。右：每种模式三条 100 跳链的整链耗时。柱为三次运行值的中位数；点为单次运行。

最终区块体和分片 opening，输入数为 1、2、4 和 8，跨越两个真实分片。完整测试套件和静态检查在该系列后期快照上于 Windows 和 macOS 通过，竞态检测器在 macOS 上通过成员、历史表示和委员会运行时各包，完整套件和静态检查在经审计的提交上再次通过。

计时与汇总。认证接收延迟一般从钱包第一次实际发出 HTTP 请求起，到收款方验证并完成本地原子记录止。资本与源交易失败的驱动程序包含少量发送前的构造，所以其比较仅限于同一驱动程序族内。本地 NoSync 事务完成并不是同步持久化的回执。公共与成员完成时间戳记录的是对经认证执行的观察，而不是委员会的内部提交。发送方的单调时钟测量跨站点 ACK 延迟，包括返回路径，收款方可以在前一个发送方收到该 ACK 之前就发起其后继交易。目标提供速率、实际发送速率和含排空的完成速率分别保留。图注区分交易分位数与跨运行的统计量，单次运行内的样本变异不是对跨运行不确定性的估计。

交付门控运行。在归档的交付门控版本上，有门控的变体等待三名成员 INSTALL 完整材料，或等待经认证的公共执行成功，以先发生者为准，且不收集第二个签名法定人数。在每秒 200 笔、持续 30s 的条件下，每种模式三次配对运行各发送并完成 6,000 笔支付。运行级接收 P50 的中位数在立即投递时为 1.300 ms，有门控时为 2.167 ms。另外的 100 跳试验把中位链耗时从 287.983 降到 200.356 ms (图 S1)。

基线版本的对照。赔付与偿还矩阵中三次 5% 运行的备付金轨迹 (图 S2) 按驱动程序的释放节奏采样，所以所采样到的最低备付金不是备付金规模的结果。

三对十四服务运行，顺序依次为正常/扣留、扣留/正常、正常/扣留，使用基线版本的初始构建 (批处理结果标签分离之前)，委员会与成员数据库落盘，成员和钱包为 NoSync。每次以 100 笔/秒发送 4,000 笔支付，历时 40s。与之并行，另有四个两跳演示，其源交易或被正常送达 (正常运行)，或被扣留、在 30s 后被赔付并随后偿还 (扣留运行)。全部六次运行中，4,000 笔支付全部完成。各所有者角色最终持有的 CAL (30,000 和 830,000) 以及两个备付金余额 (10^{12}) 在六次运行中一致，扣留运行记录了 400 CAL 赔付和 400 CAL 偿还，所以最终分配与延迟中性一致。正常/扣留运行的接收 P50 分别为 1.070/1.127、1.089/1.108 和 1.083/1.097 ms，P95 为 26.84/59.23、29.49/58.44 和 27.41/55.15 ms；所有运行的完成速率为 97.2–97.8 笔/秒。接收 P50 比配对的正常运行高 1.3–5.3%，P95 为其 1.98–2.21 倍。因此，并发赔付在该配置下影响接收的尾部，而这些配对不能区分造成这一影响的具体环节。

六次各 1,000 笔支付、速率 100 笔/秒的运行，使用九服务配置 (委员会落盘，钱包为 NoSync)，仅在

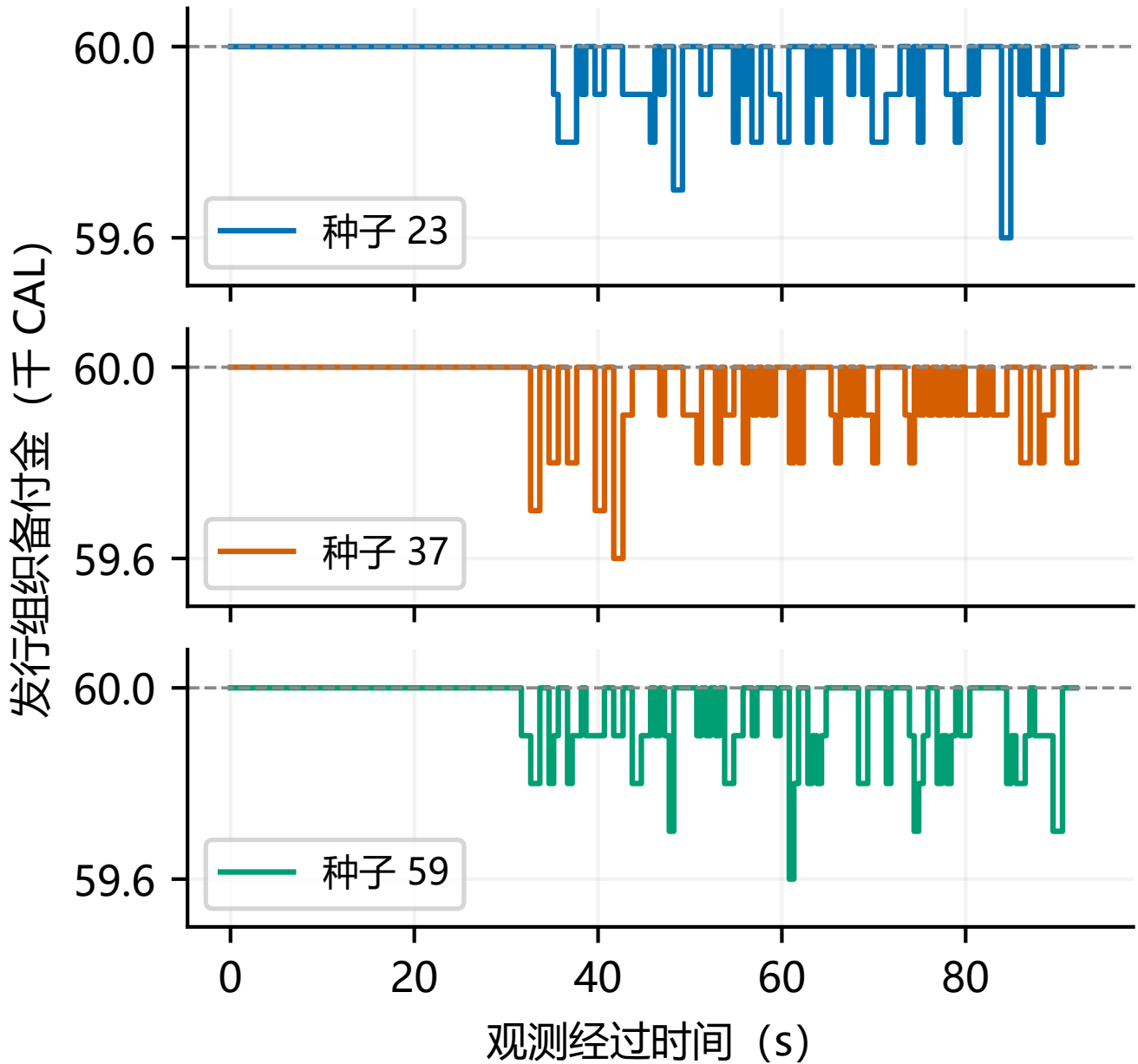


图 S2. 三次 5% 运行（每次 60 个被赔付的源交易）的备付金采样轨迹。偿还使初始的 60,000 CAL 得到恢复。纵轴限定在 59,600–60,000 CAL，以显示暂时的占用。采样点反映驱动程序的释放节奏。

成员存储上不同。NoSync 成员时，接收 P50 为 1.002–1.035 ms，P95 为 33.97–40.62 ms，含排空的完成速率为 93.11–93.93 笔/秒。成员提交为同步时，P50 为 43.19–51.85 ms，P95 为 80.98–96.56 ms，完成速率为 90.99–92.72 笔/秒。全部支付完成。由于钱包仍为 NoSync，这一对照单独隔离出同步成员提交的代价。

历史表示微基准测试与批处理集成。这些结构性测量使用早期的耦合历史表示命令，它同时还承载经济步骤。在 Windows amd64、Go 1.27.1 上的微基准测试，把 $k \in \{1, 2, 4, 8\}$ 个缺失输入放在同一个历史区块的同一个分片中。五次重复中，单条命令需要 k 条命令和 k 次分片适配，而一个批处理只需各一次，输入适配仍为 k 次。在 $k = 8$ 时，编码命令总字节数从 127,336 降到 2,752（图 S3）。分散对照把批处理与紧凑编码区分开：八个独立的目标区块仍需要八条命令和八次分片适配，而字节数从 30,008 降到 5,440。在 $k = 1$ 时，两

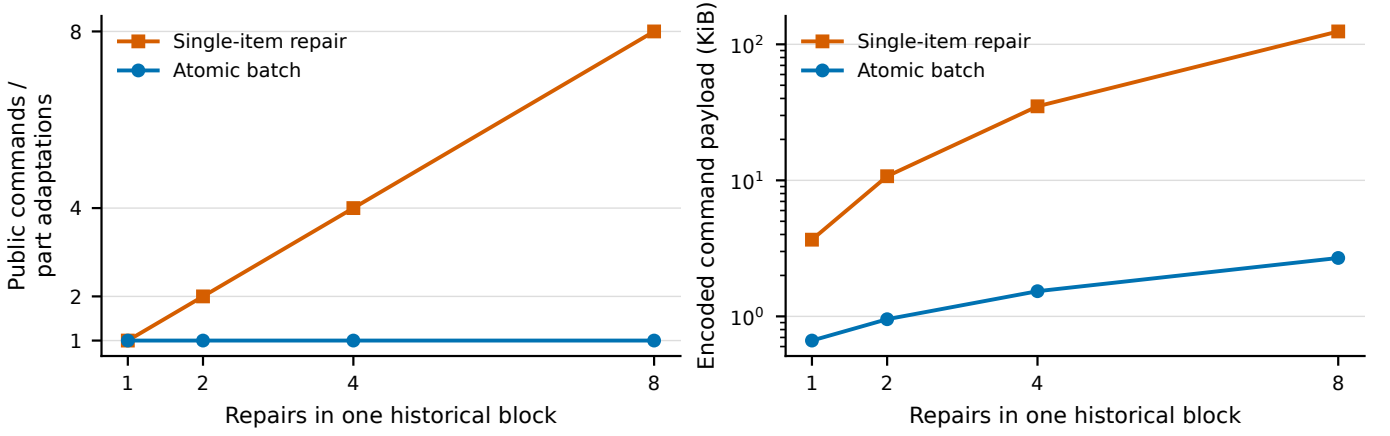


图 S3. 早期耦合命令在同一区块、同一分片内的历史表示开销与义务数 k 的关系。两个子图分别显示命令/分片适配次数，以及编码命令负载总量 (KiB, 对数刻度)。每种配置五次重复，得到相同的结构性测量。输入适配仍为 k 。

种格式都执行一次分片适配，编码大小分别为 3,751 和 680 字节。因此，紧凑字段解释了字节数下降的一部分，共享的区块表示则解释了适配次数的减少。这些是编码后的命令大小，不是网络流量总量或密码学计算总量。它们也不包含 decision-first 版本中每个已决定输出所携带的那条 88 字节的决定命令。

一次归档的、十四服务落盘的批处理集成运行 (f82f8cd) 把四个实际缺失的源交易合并为一个 1,568 字节的批处理，其 BatchID 在全部委员会副本上一致。迟到父交易的处理之后，随后以 100 笔/秒提供的 1,000 笔支付全部完成。接收 P50/P95 为 1.048/26.531 ms，总完成时间为 11.120 s，含观察与关停。缺口和 outbox 都已排空，委员会状态哈希一致。该批处理携带经济步骤，且早于决定与历史表示的分离。

外部出资的授权控制器。资本实验每 200 ms 采样一次成员可用额度。令 ρ 为声明的每秒 CAL 授权需求，其中把每个工作负载单元里的父请求和子请求都计入，令 u 为认证尚未完成的不同请求数。控制器设定

$$b = 200 + 100u,$$

$$H_{\text{low}} = \rho(0.8 \text{ s}) + b,$$

$$H_{\text{target}} = 1.25H_{\text{low}}.$$

其中水位线和突发余量以 CAL 计。当观察到的最低成员可用额度 A_{min} 低于 H_{low} 时，建议的授权额度增量为

$$\Delta B = 100 \left\lceil \frac{1.5(H_{\text{target}} - A_{\text{min}})}{100} \right\rceil,$$

并受已配置的授权上限和可用的外部资金限制。同一时刻至多有一次增量在途。其公共命令原子地扣减外部来源，并记入备付金和授权额度，跟随的成员加上由此产生的份额之差，而不清除已有批准。该实验预先提供 ρ ，而不是估计未知的生产需求。控制器的决定是对实际有资金支撑的转账的请求，而不是创造支撑资本的机制。

资本负载记账。每个条件各有一次 300-s 运行，每个成员一个 Worker，用户 FUEL 充足。从 3,600 CAL 起步，恒定、速率阶跃和父交易延迟三个条件在外部转入 10,800、18,000 和 19,700 CAL 之后，分别结束于 14,400、21,600 和 23,300 CAL (表 S7)。每个条件都完成 6,000 个两跳单元，即 12,000 笔支付，花费 1,128,000 用户 FUEL。相应的固定高值对照全部完成，而固定低值条件只关闭了 6,000 个单元中的 46 个，有 5,826 个未启动，并记录了 64 笔部分批准的支付。这些类别的单位不同，不合并为一个支付计数。逐次运行表保留了完整的七条件比较，但并不意味着每个条件重复三次。该系列在较早的版本上运行，其中没有发生赔付。

表 S7

实际的资本新增。每个条件各有一次 300-s 运行，起始为 3,600 CAL，完成 12,000 笔支付。两列均以 CAL 计；每次新增都扣减一个外部出资账户。

负载	最终备付金	新增资本
恒定：20 单元/秒	14,400	10,800
阶跃：10 至 30 单元/秒	21,600	18,000
父交易延迟 3 s	23,300	19,700

身份、资源使用与保留的历史。多身份配置使用共享的客户端，所以 2,048 个钱包身份并不对应 2,048 个独立的操作系统进程。把独立输入与十跳支付相比，三次运行的中位数为：每笔支付 12.67 对 15.24 core-ms，每笔支付 59.32 对 71.83 KiB 的 HTTP 负载。内存报告把节点总 RSS 和保留的历史，与活跃义务、待处理工作和 outbox 分开，所以不断增长的历史占用不会被报告为不断增长的支付积压。逐次运行的资源汇总及其归档源位置包含在制品索引中。

针对特定端点的 Lightning 参照。归档部署使用 LND v0.21.3-beta 和 regtest 中的 Bitcoin Core 31.1，采用同步的 LND 持久化及其 50-ms 通道提交设置。三次全新部署中，每次有两个节点和一条直连通道。经过 20 次预热转账之后，每次运行发出 100 笔串行、交替的 10,000-satoshi 支付，使用预先准备的 invoice，并以接收方 SETTLED 作为完成事件；所报告的 P50/P95 为 282.12/320.17 ms。另有一次 Next 运行，以 100 笔/秒发送 300 笔支付，持续 3 s，使用内存中的服务状态和 NoSync 钱包存储。收款方认证接收的 P50/P95 为 0.9625/1.2075 ms。工作负载并发度、持久性和完成语义各不相同，所以这些记录是部署参照，而不是归一化的加速比。

A. 归档的运行范围系列

归档的运行范围系列是对续花结果的补充。一个活跃的组织使用赞助的费用、复用的地址和独立的最终输入。五次各 150,000 笔支付的运行全部完成（图 S4）。在 2,000 笔/秒的目标下，两次运行的完成速率为 1,948.66–1,950.45 笔/秒（含排空），接收 P50 为 2.521–2.566 ms。把目标提高到 2,200 时，完成速率为 2,103.63–2,106.29 笔/秒。在 2,400 时，实际达到的速率为 2,174.68，P50/P95 升至 7.174/89.38 ms，4,096 项的待处理准入上限被触发 1,635 次。运行曲线把达到的完成速率与其对应的队列压力配对给出。

S4. 旧构建的实验结果

以下结果保留原构建、存储与测量条件，属于旧版本证据；当前正文采用最终构建 721800c 的三组实验。旧版性能与 CPU 测量来自不同构建和负载，不能混用为同一系统的容量推导。

A. 无需逐跳结算的连续花费

我们把认证续花与在各跳之间等待公共执行进行比较。在基线版本上，100 跳链到达最后一次认证接收的中位耗时为 158.846 ms，而每一跳都等待公共确认时为 65.204 s。三个钱包轮流转账 100 CAL，每笔支付都由其前一笔的输出在线构造，并用自己的最终 FUEL 支付费用。每种模式重复三次，在九服务的内存配置上共得到六次运行、600 笔支付。两种模式都以最后一次认证接收为终点，等待模式则在各跳之间增加一次公共确认。在委员会 commit、flush 和 gossip 设置分别为 250、10 和 10 ms 时，公共执行及其观察每跳耗时约 0.65 s。认证续花消除了收款方路径上的这段等待。

表 S8

赔付决定记录与历史表示命令 (C3)。二者都是只提交一次的公共命令；区别在于已存储的历史所显示的内容。

	赔付决定记录	赔付决定加历史表示 (C3)
经济关闭	备付金扣减、义务状态和费用效果在一项公共决定中一并提交，与适配无关；后继交易不变。	同一项决定；历史表示命令不增加经济效果。
已存储的历史区块体	原始区块体不变；已消费输入的资金来源从决定索引读取。	已存储的区块体指明备付金扣减；交易身份、所有者授权和完整区块身份 (H_b, P_b) 保持不变并经验证。
检查指定的历史输入	需要把决定索引与区块对照查阅。	直接从区块本身读取资金来源，并对照原始承诺检查。
读取方	命令历史和决定记录的任何读者。	历史表示的构造、安装和本地检查读取修订后的区块体；钱包、同步和重放读取原始命令。
存储与重放	原始区块加决定记录；重放运行原始命令，其中包括决定。	另外还有修订后的区块体、修订版本和任务；重放运行相同的原始命令以及历史表示命令；同一区块的命令共享分片适配，安装推进到最新已提交的修订版本。
进展依赖	决定的公共执行。	三份适配贡献和一个当前修订版本；经济关闭不依赖于它们。
状态	已作为经济路径实现；在 decision-first 运行中有测量。	已实现；见上文的功能运行与回归测试。

图 S5 给出六次整链耗时。快速运行耗时 158.492–159.905 ms。快速模式下全部 297 个后继请求，都消费其前一笔已认证的输出，并在该输出的认证接收之后、其公共完成被观察到之前发出。一个单输入、单输出的 TXCer 为 779 字节，请求在第一跳为 1,979 字节，之后为 2,766 字节。对于这种交易形态，直接证据的大小在全部 100 跳中保持不变，因为每个请求携带的是其直接输入的证书，而不是祖先链。

为检验这一优势是否依赖于内存中的成员存储，我们在九服务的落盘配置上，使用 NoSync 钱包和用户自付 FUEL，做了三组配对的 100 跳试验。这些试验仅在成员存储上不同 (NoSync 或同步；顺序为 N/S、S/N、N/S)，每组之后各跟一条同步的等待链。中位耗时分别是：NoSync 续花 681.127 ms，同步续花 4.471 s，同步等待 65.013 s。因此在成员提交为同步时，认证续花仍比等待快 14.5 倍。全部 900 笔支付完成，且各委员会状态一致。

在归档的交付门控版本上，认证之后有一个投递门控，它等待三名成员 INSTALL 完整材料，或等待经认证的公共执行成功。该门控不收集第二个签名法定人数；改为立即交付会缩短续花。在每秒 200 笔支付、持续 30s 的条件下，每种模式三次配对运行各完成 6,000 笔支付，运行级接收 P50 的中位数从有门控时的 2.167 ms 降到 1.300 ms (下降 40.0%)。另外的 100 跳试验从 287.983 降到 200.356 ms (下降 30.43%)。授权不变，补充材料列出了配对运行。

B. 有资金支撑的关闭与偿还

本实验检验赔付是否保持后继支付不变，以及每次赔付是否在其源交易随后执行时恰好偿还一次。

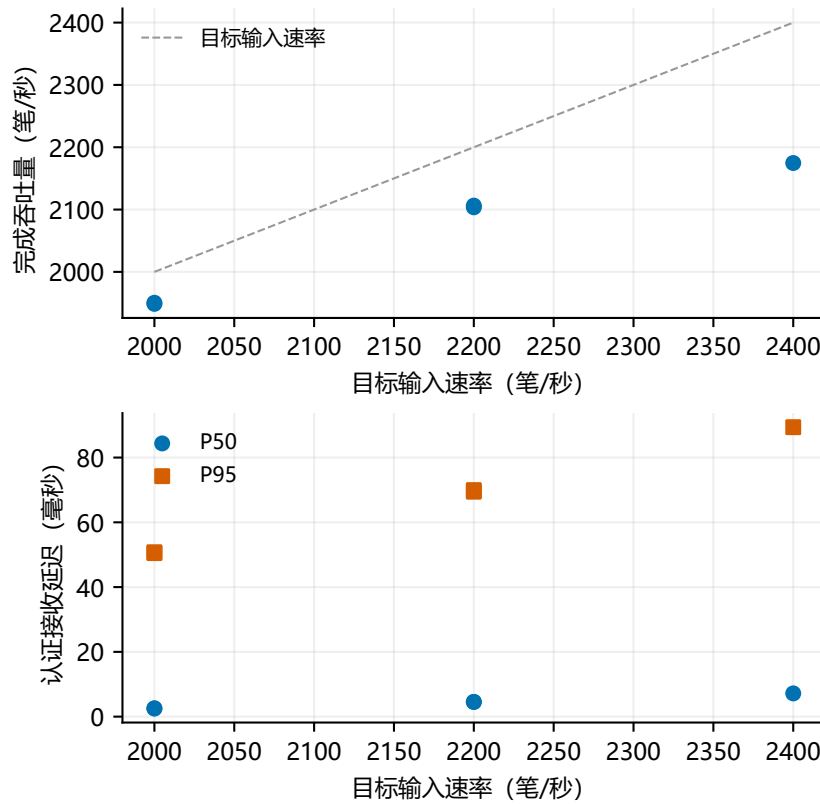


图 S4. 运行点与目标提供负载的关系。上：完成吞吐量（笔/秒），含排空。下：接收延迟（ms）。每个点是一次 150,000 笔支付的运行：目标 2,000 两次，目标 2,200 两次，目标 2,400 一次。P50/P95 汇总的是单次运行内的交易，不是跨运行的不确定性。所有服务共享一台主机。

赔付与偿还矩阵（基线版本）。九次运行使用随机种子 23、37 和 59，被扣留父交易的比例分别为 0%、1% 和 5%。每次运行按每秒 20 个两跳单元的速率提供负载，持续 60 s（1,200 个单元，2,400 笔支付），对应一份固定的 60,000-CAL 备付金，使用用户自付 FUEL 且不充值。九服务部署使用落盘的委员会应用状态和 BlockStore，成员和网关在内存中，钱包为 NoSync。驱动程序只有在每个被扣留父交易的赔付已在全部四个副本上提交并完成安装之后，才释放该父交易，所以每个被扣留的源交易都在 30-s 公共时间期限之后被赔付，并在其后执行。

全部 21,600 笔支付都完成关闭。216 次赔付（1% 的每次运行 12 次，5% 的每次运行 60 次）各自之后都有偿还。每次运行结束时，没有义务处于 Open 或 Repaired，缺口为零，净 Spent 和 Reserved 为零，备付金回到 60,000 CAL（表 S9）。所采样到的最低备付金在 1% 时为 59,800–59,900 CAL，在 5% 时为 59,600 CAL。它反映的是驱动程序的释放节奏，不是备付金规模的结果，补充材料绘出了三次 5% 运行的轨迹。正常子支付的运行级接收 P50 为 1.185–1.251 ms。从期限起算，委员会提交（在该版本中它承载历史表示）的观察时间中位数为 1.003–1.254 s，全部四个副本上完成修订区块体物理安装的时间中位数为 1.257–1.761 s（P95 最高 2.015 s）；两者都包含 250-ms 的观察轮询。每笔正常支付花费 94 FUEL，每次赔付增加 5，奖励、销毁与退款之和等于费用上限，Held 为零。

账本层面的回归测试直接覆盖到达顺序：四跳链的全部 24 种到达顺序，在有到期赔付与无到期赔付两种情形下，各取三种签发组织序列，共得到 144 条轨迹；另有 48 条拆分-合并轨迹，在两种赔付模式下，改变一个 40/60 拆分、两条分支及其合并的全部到达顺序。192 条轨迹中的每一步都保持独立计算的资产投影

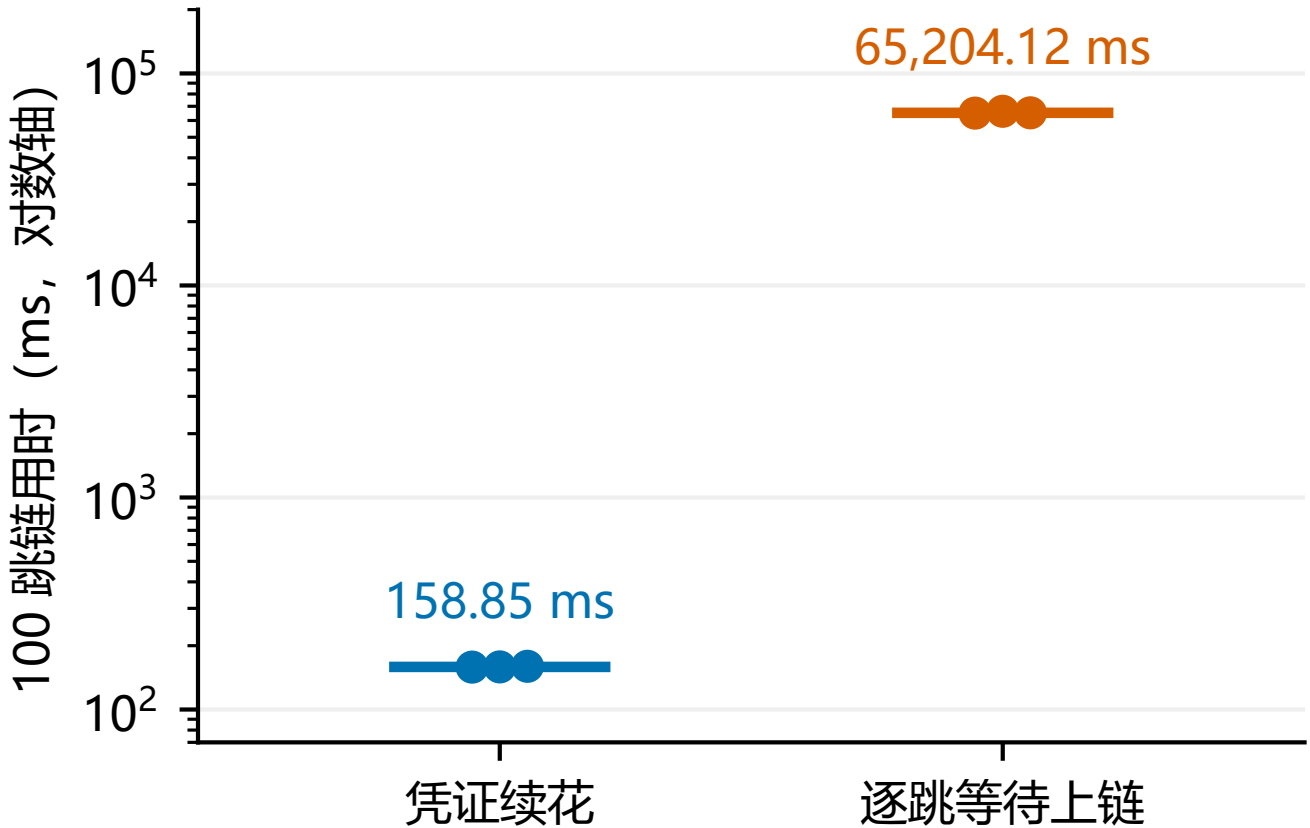


图 S5. 基线版本上 100 跳链到最后一次认证接收的耗时。圆点为每种模式的三次独立运行；横线标出它们的中位数。对数时间轴在两种模式中包含相同的终点。只有等待式基线要求在相邻支付之间进行公共确认。

守恒。每条轨迹结束时，各签发组织的备付金恢复，Spent 与 Reserved 为零，Paid = Rec，已消费的输入仍保持已消费。

两个十四服务的对照实验区分永久损失与延迟偿还。父交易始终不到达时，孙支付保留其 100 CAL，而负责的签发组织的备付金仍然低 100 CAL。当跨组织链以孙支付最先到达的顺序到达时，中间源交易关闭其自己的直接义务，根签发组织被赔付的 100 CAL 在根交易其后执行时得到偿还。两种情况下四个委员会状态一致，缺席的父交易自身的费用和工作预留可能保持未清偿。

三组配对的十四服务运行使用基线版本的初始构建（在批处理结果标签分离之前），每次 4,000 笔支付、速率 100 笔/秒，其正常运行与扣留源交易运行最终的所有者持有和备付金余额完全一致，与主文的 CAL 延迟中性定理一致。并发赔付使接收 P95 升高为 1.98–2.21 倍，这些配对不能区分造成这一升高的具体环节。

一个使用 NoSync 钱包的存储对照把成员提交单独隔离出来：NoSync 成员时接收 P50 为 1.0 ms，同步提交时为 43–52 ms。

早期版本的资本系列（每个条件一次运行、没有赔付）表明，10,800–19,700 CAL 的外部新增，足以维持从 3,600-CAL 起步、各含 6,000 个两跳单元的 300-s 运行，而固定 3,600-CAL 的对照只关闭了 6,000 个单元中的 46 个。

表 S9

赔付与偿还矩阵的收尾审计。每一列都被三个随机种子复现；每次运行有 2,400 笔支付。备付金数值以 CAL 计，费用数值以千 FUEL 计，费用上限为各笔支付最大值之和。

被扣留父交易比例	0%	1%	5%
赔付 / 偿还次数	0	12	60
赔付 / 偿还 CAL	0	1,200	6,000
最终备付金	60,000	60,000	60,000
未关闭 / 净 Spent	0	0	0
费用上限总额	2,400	2,400	2,400
Rewards	201.600	201.660	201.900
Burned	24.000	24.000	24.000
Refunded	2,174.400	2,174.340	2,174.100
Remaining Held	0	0	0

C. 跨注册组织的续花

为测量通信距离的影响，我们在补充材料所记录的一个归档版本上运行了双组织套件：两个钱包进程各持 32 个身份，经 64 个 lane 发送，每个组织提供 50 笔/秒，24 次运行共完成 342,000 笔支付。延迟比较使用十跳的同组织与跨组织负载，注入的 HTTP RTT 为 0、20 和 100 ms，每个条件重复三次。每个代理方向增加一半的 RTT，组织内部认证和委员会 P2P 不注入延迟。

对共同的前 120-s 样本组，三个运行级跨组 ACK P50 的中位数依次为 1.632、21.658 和 101.967 ms，同组数值范围为 1.615–1.744 ms（图 S6）。跨组 ACK P50 随注入 RTT 变化，测得的超出部分为 1.6–2.0 ms。ACK 包含返回路径，但收款方可以在接受之后即刻发起其后继交易，无需等待该 ACK 到达前一个发送方。在 100-ms 跨组 RTT 下，三次 300-s 运行各完成 30,000 笔支付；待处理峰值为 106–110，并能排空，没有持续的尾部增长。该实验在一台物理主机内隔离出 HTTP 路径延迟。

D. 运行容量与服务可用性

独立输入吞吐（基线版本）。三次各 30,000 笔支付的运行以 2,000 笔/秒为目标，使用九服务的内存配置、NoSync 钱包和经授权的组织代付 FUEL。成员进程使用 200% 的 Go GC 目标和 16 个运行时处理器。全部支付完成，四个委员会状态一致。含排空的完成速率为 1,750.27、1,820.91 和 1,887.31 笔/秒，历时 15.9–17.1 s，接收 P50 为 2.646、2.842 和 2.824 ms，P95 为 47.9、49.1 和 48.0 ms。驱动程序记录到有 6,551–7,570 笔支付需要等待快速接收准入名额，P95 派发滞后为 179–229 ms，所以提供的负载并不是均匀的 2,000 笔/秒。这些是基线版本上的短时完成测量，没有在 decision-first 版本上重新测量；该版本在普通支付上的开销由上文的六次运行对比检验。补充材料说明了 150,000 笔支付系列所用的版本和时长。

在归档版本上，冗余认证与已 INSTALL 的证据在所述故障下维持服务。九次每秒 200 笔的运行完成 216,000 笔支付，其间有一个成员被延迟或暂停 30 s。网关丢失对照中，一名成员持有经 INSTALL 复制的完整材料，备用提交在 INSTALL 之后 2.503–2.508 s 出现。冲突对照既没有出现相互冲突的双 QC，也没有重复的本金或费用效果，而部分批准仍然是一种额度代价。

补充材料第 S2 节的多身份测量报告每笔支付的 CPU 和 HTTP 负载开销；2,048 个身份共享客户端。历史内存增长与积压分别报告。该节还给出 LND regtest 参照，其完成事件、持久化设置和负载并发度与 Next 的测量不同。

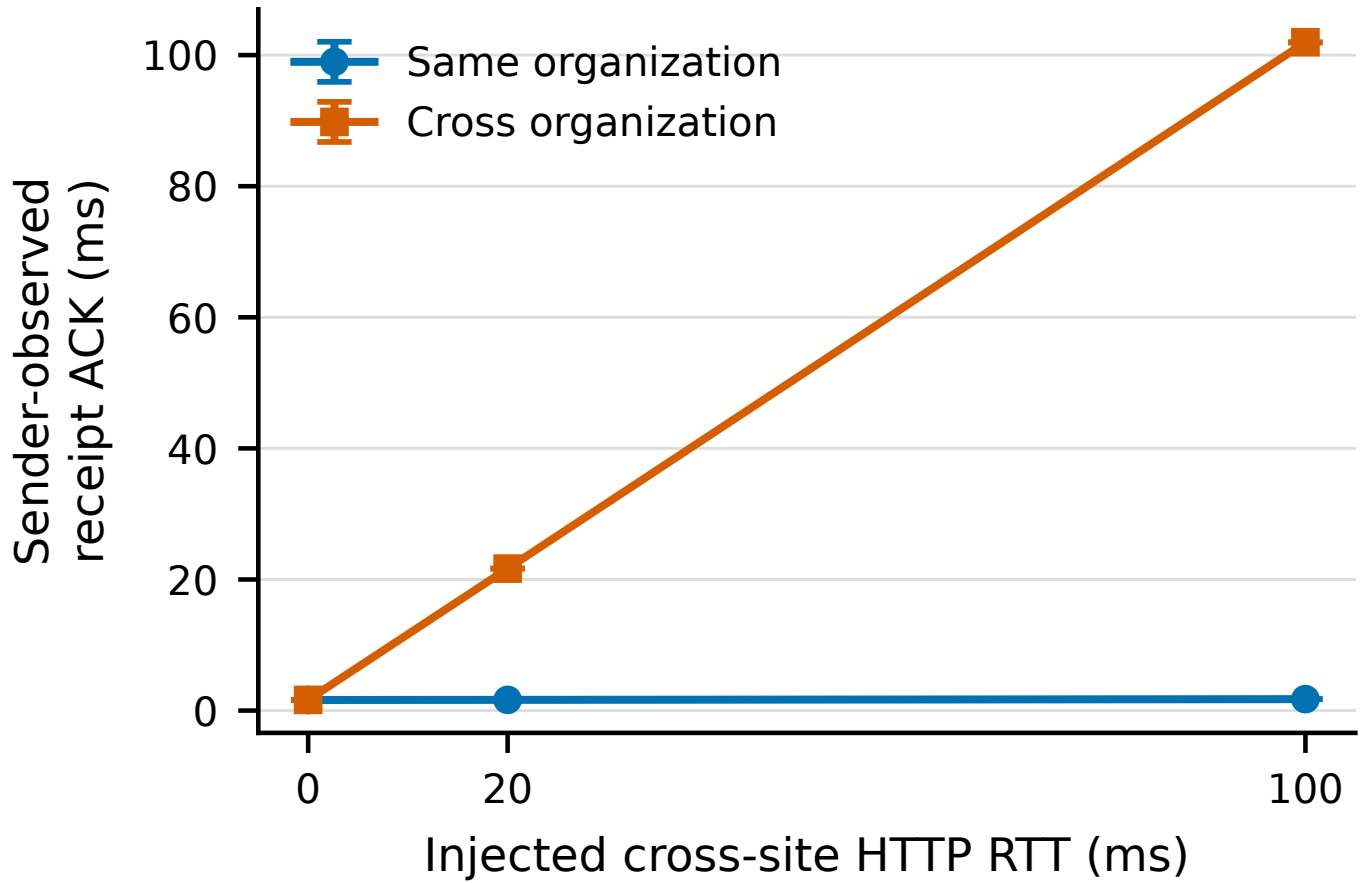


图 S6. ACK 延迟 (ms) 与注入的跨站点 HTTP RTT (ms) 的关系。每个点是前 120-s 样本组的三个运行级 P50 的中位数；误差条跨越这些 P50 的最小值与最大值。总提供速率为 100 笔/秒。委员会 P2P 未加延迟。

S5. 逐轮实验记录

表 S10

归档的单组织运行点实验。每轮发送并完成 150,000 笔付款。实际发送速率与完成速率的单位为笔付款/s，认证接收分位数的单位为 ms。完成速率计入最终后台排空时间。峰值表示后台处理尚未完成的付款数峰值；上限触发次数表示 4,096 笔未完成付款上限的触发次数。

轮次	目标	实际发送	完成	P50	P95	峰值	上限触发次数
1	2000	1,964.53	1,948.66	2.521	50.499	2159	0
2	2000	1,969.65	1,950.45	2.566	50.761	2205	0
3	2200	2,127.70	2,103.63	4.538	69.463	2655	0
4	2200	2,127.15	2,106.29	4.553	69.945	2548	0
5	2400	2,206.95	2,174.68	7.174	89.376	4096	1635

表 S11

来源回收基线矩阵的逐轮记录。每轮在 60 秒内提供 1,200 个两跳单元（2,400 笔付款）。收款分位数为正常子付款的毫秒数；总耗时及截止至全部副本物理安装的观察中位数为秒，后者含 250 毫秒轮询。每轮结束时备付余额为 60,000 CAL，净支出及缺口为零。

种子	扣留率	赔付/回款	总耗时	子 P50	子 P95	安装 P50
23	0%	0/0	61.540	1.251	1.620	—
23	1%	12/12	90.225	1.185	1.432	1.759
23	5%	60/60	90.096	1.191	1.536	1.257
37	0%	0/0	61.595	1.199	1.559	—
37	1%	12/12	91.935	1.190	1.479	1.509
37	5%	60/60	91.530	1.192	1.426	1.257
59	0%	0/0	61.550	1.191	1.461	—
59	1%	12/12	90.595	1.193	1.452	1.761
59	5%	60/60	90.346	1.194	1.494	1.507

表 S12

归档的有限备付实验。每个条件进行一轮 300-s 运行，资金数值单位为 CAL。已关闭计数以两跳单位计。自适应增加来自外部资金账户的实际转账。均值是对 300 秒输入窗口内采样余额进行积分所得的组织 CAL 余额时间平均值。

条件	初始	新增	最终	均值	已关闭
恒定 / 固定低备付	3,600	0	3,600	3,600.0	46/6000
恒定 / 固定高备付	14,400	0	14,400	14,400.0	6000/6000
恒定 / 自适应	3,600	10,800	14,400	13,801.8	6000/6000
阶跃 / 固定高备付	21,600	0	21,600	21,600.0	6000/6000
阶跃 / 自适应	3,600	18,000	21,600	14,078.7	6000/6000
延迟 / 固定高备付	28,800	0	28,800	28,800.0	6000/6000
延迟 / 自适应	3,600	19,700	23,300	20,611.2	6000/6000

表 S13

同步成员续花对照：每轮 100 跳，九服务磁盘配置，用户自付 FUEL，钱包 NoSync。单位为秒，终点为末跳认证收款。N/S 表示两种快速模式的配对顺序，每对之后运行同步等待链。

轮对	顺序	NoSync 续花	同步续花	同步等待
1	N/S	0.681127	4.492812	64.969480
2	S/N	0.705604	4.309984	65.193522
3	N/S	0.636821	4.470844	65.012947
中位数		0.681127	4.470844	65.012947